

**FECAP**

Ciência da Computação – 5º semestre

## Projeto Interdisciplinar: Inteligência Artificial



### Concepção do Produto

Sistema de Classificação e Contagem Inteligente de Alimentos com Visão Computacional para a Lideranças Empáticas

#### **Integrantes:**

Pedro Lemos – RA: 23025380 · André Gregório – RA: 24026489 ·  
Yan Cezareto – RA: 24026005 · Guilherme Fogolin – 24026241

#### **Professor:**

Rafael Diogo Rossetti

Março 2026

---

## Stack Tecnológica

Resumo das principais tecnologias, frameworks e bibliotecas que estão previstas no projeto AbraceAI (Sujeita a alterações).

| Camada              | Tecnologias                                                                                                                                     |
|---------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| Frontend            | React, Vite, Tailwind CSS, Shadcn/UI, Zustand, TanStack Query, Socket.io-client, @hello-pangea/dnd, Recharts, @react-pdf/renderer, React Router |
| Backend             | FastAPI (Python), Socket.io (server), Uvicorn                                                                                                   |
| Visão Computacional | YOLOv8 (Ultralytics), OpenCV (cv2), ByteTrack / BoTSORT, PyTorch, scikit-learn                                                                  |
| Banco de Dados      | PostgreSQL (Azure Database for PostgreSQL)                                                                                                      |
| Armazenamento       | Azure Blob Storage (evidências visuais)                                                                                                         |
| Infraestrutura      | Docker, Azure Container Apps, GitHub Actions (CI/CD), Git + GitHub                                                                              |

---

# 1. Introdução

## 1.1 Contexto

O Lideranças Empáticas (LE) é uma iniciativa que une impacto social e educação empreendedora. Por meio da arrecadação de alimentos e recursos financeiros, alunos de graduação desenvolvem ações práticas que aplicam conceitos de gestão, liderança e organização. A dinâmica central do LE envolve uma competição solidária entre equipes de alunos, onde cada equipe é responsável por arrecadar alimentos doados junto à comunidade. Os resultados de cada equipe são comparados ao longo de ciclos de arrecadação, estimulando o engajamento e o senso de responsabilidade coletiva.

## 1.2 Problema Operacional

Atualmente, o registro das doações arrecadadas pelo LE é realizado manualmente em planilhas, contabilizando apenas o peso total (por exemplo, "1 tonelada de alimentos doados"). Esse método apresenta limitações significativas:

- **Falta de granularidade:** não há distinção entre tipos de alimento (arroz, feijão, macarrão, etc.), impossibilitando análises detalhadas sobre a composição das doações.
- **Contagem por equipe imprecisa:** sem um sistema padronizado, a atribuição correta da arrecadação a cada equipe depende de processos manuais suscetíveis a erro.
- **Ausência de rastreabilidade:** não existe registro visual ou histórico detalhado que permita auditoria posterior ou verificação de inconsistências.
- **Processo lento:** a contagem manual de centenas de itens consome tempo e recursos que poderiam ser direcionados para a operação do projeto.

Essas dificuldades comprometem a confiabilidade dos dados de arrecadação e a transparência da competição entre as equipes, elementos fundamentais para manter o engajamento dos participantes e a credibilidade da iniciativa.

## 1.3 Proposta de Solução

O AbraceAI é um sistema web de classificação e contagem inteligente de alimentos, baseado em Visão Computacional e Inteligência Artificial. A solução utiliza a câmera do computador para capturar itens alimentícios embalados em tempo real, identificando automaticamente o tipo de produto e registrando a contagem por equipe e por categoria. O sistema substitui o processo manual por um fluxo automatizado, rápido e auditável, garantindo dados granulares e confiáveis para a gestão da arrecadação do LE.

---

## 2. Objetivo do Sistema

### 2.1 Objetivo Geral

Desenvolver uma solução integrada (captura via câmera, Visão Computacional/IA, backend em nuvem e aplicação web) para classificar e contar pacotes de alimentos arrecadados, registrando automaticamente a contagem por equipe arrecadadora e por categoria de alimento.

### 2.2 Objetivos Específicos

- Definir requisitos e fluxo de operação completo, incluindo seleção de equipe, início e encerramento de sessão de contagem, validação e auditoria dos registros.
- Coletar e rotular imagens representativas dos pacotes de alimentos, contemplando variações de marcas, tamanhos e orientações dos produtos.
- Implementar o pipeline de Visão Computacional com pré-processamento de imagem, detecção e isolamento da região de interesse (ROI) e tratamento de variações de iluminação.
- Treinar e avaliar um modelo de IA (YOLO com fine-tuning) para classificar categorias de alimentos, avaliando o desempenho com métricas quantitativas e matriz de confusão.
- Implementar estratégia de contagem sem duplicidade, combinando tracking de objetos, zona de detecção/saída e cooldown temporal.
- Persistir eventos de contagem no backend (equipe, categoria, data/hora, confiança e evidência visual) em banco de dados relacional.
- Entregar interface web funcional para operação (seleção de equipe, scanner em tempo real, visualização de contagens, histórico e geração de relatórios).
- Implantar a solução em nuvem (Azure) com segurança, logs e documentação de arquitetura.

### 3. Escopo do MVP

O MVP (Minimum Viable Product) do AbraceAI contempla as funcionalidades essenciais para validar a proposta junto ao Lideranças Empáticas.

#### 3.1 Incluído no MVP

| Funcionalidade       | Descrição                                                                                                         |
|----------------------|-------------------------------------------------------------------------------------------------------------------|
| Home Page (Kanban)   | Visualização dos grupos em cards estilo Kanban, com resumo de arrecadação por grupo e opção de expandir detalhes. |
| Gestão de Grupos     | Criação, edição e remoção de grupos de arrecadação.                                                               |
| Tela de Scanner      | Captura em tempo real via câmera com detecção e classificação automática de itens alimentícios embalados.         |
| Contagem Inteligente | Registro automático com tracking anti-duplicidade, correção manual pelo operador e salvamento de evidências.      |
| Relatórios PDF       | Geração de relatório consolidado com arrecadação por grupo e por categoria.                                       |
| Perfis de Usuário    | Operador, Coordenação e Admin com permissões distintas.                                                           |
| Backend em Nuvem     | API REST + WebSocket, banco PostgreSQL e armazenamento de evidências.                                             |

#### 3.2 Fora do Escopo do MVP (Extensões Futuras)

- Esteira automatizada com linha de passagem e controle de velocidade.
- Inferência em edge (dispositivo dedicado) para reduzir latência e dependência de rede.
- Dashboard com ranking de equipes, metas e séries temporais.
- Detecção de condições inadequadas (iluminação fora do padrão, item parcialmente visível) com alertas ao operador.
- Ampliação massiva de classes e retreinamento incremental automatizado.

---

## 4. Público Usuário

O AbraceAI define três perfis de usuário com responsabilidades e permissões distintas.

### 4.1 Operador

Perfil principal de uso no dia a dia. O operador é a pessoa que posiciona os itens alimentícios na câmera e conduz a sessão de contagem. Suas atribuições incluem: selecionar o grupo arrecadador ativo, iniciar e encerrar sessões de contagem, operar o scanner em tempo real, corrigir classificações incorretas (antes ou após o registro) e visualizar a contagem parcial da sessão.

### 4.2 Coordenação

Perfil de supervisão e acompanhamento. A coordenação monitora o andamento da arrecadação e gera relatórios. Suas atribuições incluem: visualizar a home page com o panorama de todos os grupos, consultar o histórico de contagens por equipe e por período, gerar e exportar relatórios em PDF e auditar registros com base nas evidências visuais salvas.

### 4.3 Administrador (Admin)

Perfil de gestão do sistema. O admin configura e mantém o ambiente operacional. Suas atribuições incluem: cadastrar e gerenciar equipes e usuários, atribuir perfis e permissões, configurar parâmetros do sistema (classes de alimentos, limiar de confiança, tempo de cooldown) e acessar logs do sistema e métricas de desempenho.

*Nota para o MVP: na fase de validação, os próprios desenvolvedores atuarão como operadores simulando o papel de supervisor, realizando a contabilização dos itens.*

---

## 5. Jornada de Uso

A jornada de uso descreve o fluxo completo de operação do AbraceAI, desde o acesso ao sistema até o encerramento da sessão de contagem.

### 5.1 Descrição do Fluxo

#### Acesso e Seleção do Grupo

O operador acessa o AbraceAI e visualiza a Home Page no formato Kanban. Cada card representa um grupo arrecadador, exibindo um resumo da quantidade já arrecadada. O operador identifica o grupo que irá fazer a entrega e clica no botão "Iniciar Contagem" presente no card do respectivo grupo.

#### Sessão de Contagem (Scanner)

A tela do scanner é aberta e a câmera do computador é ativada. O feed de vídeo é exibido em tempo real na interface. O operador posiciona os itens alimentícios embalados, um por vez, na zona de detecção visível na tela.

#### Detecção e Registro Automático

Ao detectar um item, o modelo de IA identifica a classe (tipo de alimento) e o tracker atribui um ID único ao objeto para evitar duplicidade. Quando o item permanece estável na zona de detecção ou sai da área, um período de cooldown é iniciado. Durante esse período, a tela exibe o nome do item detectado, a confiança da predição e opções para cancelar ou corrigir a classificação. Se o operador não interagir, o item é registrado automaticamente ao final do cooldown.

#### Correção e Auditoria

Caso o modelo classifique um item incorretamente, o operador pode corrigir a classe antes do registro (durante o cooldown) ou após o registro, consultando as evidências visuais salvas (frame capturado no momento da detecção).

#### Encerramento da Sessão

Quando todos os itens forem processados, o operador clica em "Encerrar Sessão". Um resumo da sessão é exibido contendo a quantidade total de itens registrados, a distribuição por categoria e a duração da sessão. Os dados são persistidos no backend e o operador retorna à Home Page.

---

## 6. Requisitos Funcionais e Não Funcionais

### 6.1 Requisitos Funcionais

#### RF01: Gestão de Grupos (Home Page)

- O sistema deve exibir os grupos arrecadadores em layout Kanban com cards individuais.
- Deve ser possível criar, editar e remover grupos.
- Cada card deve exibir um resumo da arrecadação do grupo (quantidade total e por categoria).
- Deve haver botão para expandir detalhes do grupo.
- Deve haver botão para iniciar uma sessão de contagem vinculada ao grupo selecionado.

#### RF02: Captura e Processamento de Vídeo

- O sistema deve acessar a câmera do computador via browser e exibir o feed em tempo real.
- Os frames devem ser enviados ao backend via WebSocket para processamento pelo modelo de IA.
- O sistema deve funcionar com uma taxa mínima de processamento que permita operação fluida.

#### RF03: Detecção e Classificação de Alimentos

- O sistema deve detectar pacotes de alimentos embalados na zona de detecção da câmera.
- Cada item detectado deve ser classificado em uma categoria de alimento (inicialmente: Arroz, Feijão, Outros; com expansão futura).
- O sistema deve registrar o nível de confiança da predição para cada detecção.

#### RF04: Contagem sem Duplicidade

- O sistema deve implementar tracking de objetos (ByteTrack/BoTSORT) para atribuir ID único a cada item detectado.
- Deve utilizar zona de detecção e zona de saída para determinar quando um item deve ser contabilizado.
- Deve aplicar cooldown temporal após cada detecção para evitar recontagem.
- A combinação dessas três estratégias deve garantir que cada item físico seja contado exatamente uma vez.

#### RF05: Registro e Correção de Itens

- O registro do item deve ocorrer automaticamente após o período de cooldown, caso o operador não cancele.
- O operador deve poder cancelar o registro durante o cooldown.
- O operador deve poder corrigir a classificação de um item antes do registro (durante o cooldown) ou após o registro (via consulta de evidências).
- Cada registro deve salvar: categoria, timestamp, confiança, equipe associada e evidência visual (frame capturado).

#### RF06: Armazenamento de Evidências Visuais

- O sistema deve salvar o frame capturado no momento da detecção de cada item como evidência.
- No MVP, as evidências serão armazenadas localmente no servidor.
- Na versão final, as evidências serão armazenadas no Azure Blob Storage.
- As evidências devem ser acessíveis para consulta e auditoria posterior.

#### RF07: Geração de Relatórios



- O sistema deve gerar relatórios em PDF contendo a arrecadação por grupo, por categoria e no total geral.
- Os relatórios devem incluir ranking dos grupos, detalhamento por categoria, totais e gráficos visuais (barras, pizza).
- Deve ser possível filtrar relatórios por período/data.
- A geração deve ser acessível via botão na Home Page.

#### **RF08: Gestão de Usuários e Perfis**

- O sistema deve suportar cadastro de usuários com três perfis: Operador, Coordenação e Admin.
- Cada perfil deve ter permissões distintas conforme descrito na Seção 4.
- Deve haver autenticação para acesso ao sistema.

#### **RF09: Histórico de Sessões**

- O sistema deve registrar o histórico completo de sessões de contagem.
- Deve ser possível consultar sessões por equipe e por período.
- Cada sessão deve exibir resumo (total de itens, distribuição por categoria, duração).

## **6.2 Requisitos Não Funcionais**

| ID    | Categoria           | Descrição                                                                                                                          |
|-------|---------------------|------------------------------------------------------------------------------------------------------------------------------------|
| RNF01 | Desempenho          | Processar detecções em quase tempo real no cenário controlado, com latência de inferência documentada.                             |
| RNF02 | Qualidade do Modelo | Atingir métricas mínimas de classificação (acurácia, precision, recall, F1-score, mAP) com validação em conjunto de teste próprio. |
| RNF03 | Segurança           | Autenticação obrigatória, controle de acesso por perfil e comunicação via HTTPS.                                                   |
| RNF04 | Privacidade         | Não coletar dados pessoais desnecessários; foco exclusivo em itens alimentícios e dados operacionais.                              |
| RNF05 | Disponibilidade     | Serviço implantado em nuvem (Azure) com estratégia de backup dos dados e logs de operação.                                         |
| RNF06 | Manutenibilidade    | Arquitetura modular, código versionado em Git, documentação técnica e README atualizado.                                           |
| RNF07 | Escalabilidade      | Suportar múltiplos pontos de coleta simultâneos e crescimento do volume de evidências armazenadas.                                 |
| RNF08 | Usabilidade         | Interface intuitiva que permita operação rápida e fluida do scanner, minimizando a curva de aprendizado.                           |

## 7. Arquitetura do Sistema

### 7.1 Visão Geral

O AbraceAI segue uma arquitetura cliente-servidor com quatro camadas principais: captura (frontend), processamento IA/VC (backend), persistência (banco de dados e storage) e apresentação (interface web).

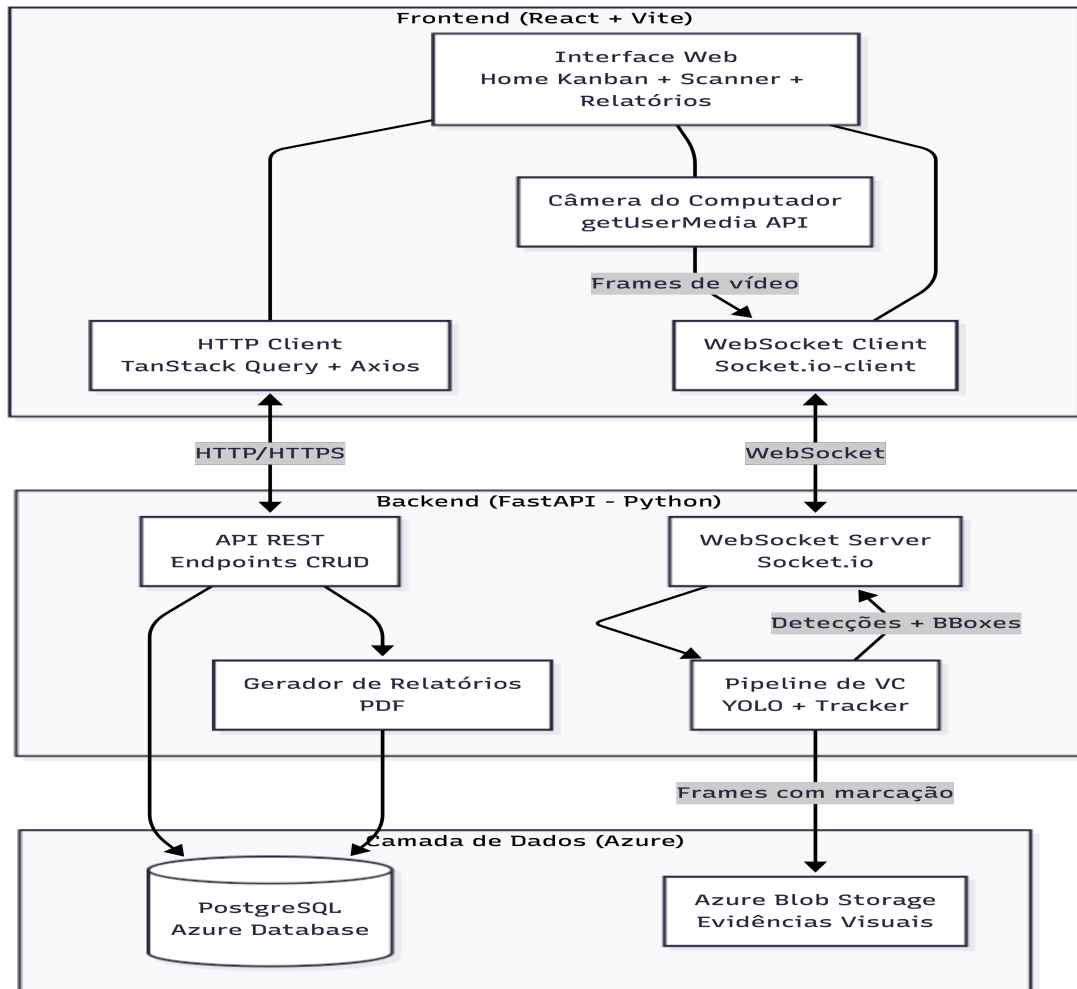


Figura 1: Arquitetura geral do sistema AbraceAI

### 7.2 Fluxo de Dados do Scanner

O diagrama de sequência a seguir ilustra a comunicação entre os componentes durante uma sessão de contagem, desde a captura do frame até o registro do item no banco de dados.

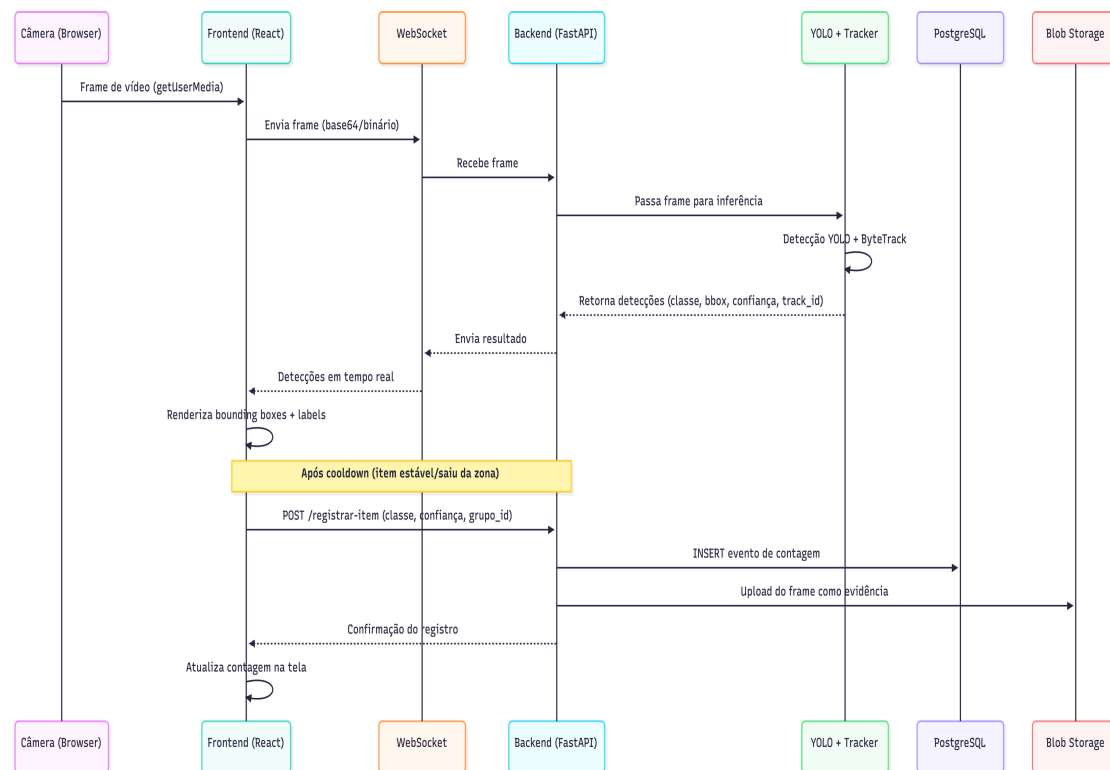


Figura 2: Diagrama de sequência do fluxo de dados do scanner

## 7.3 Detalhamento dos Componentes

### Frontend (React + Vite)

Responsável pela interface do usuário e captura de vídeo. Utiliza a API `getUserMedia()` do browser para acessar a câmera, enviando frames ao backend via WebSocket (Socket.io). Recebe as detecções processadas e renderiza os bounding boxes, labels e dados de confiança em tempo real sobre o feed de vídeo. Também consome a API REST para operações CRUD (grupos, usuários, sessões, relatórios).

### Backend (FastAPI)

Servidor centralizado que hospeda tanto a API REST quanto o servidor WebSocket. Recebe os frames do frontend, executa o pipeline de Visão Computacional (YOLO + tracker) e retorna as detecções em tempo real. Gerencia toda a lógica de negócio: sessões de contagem, registro de itens, gestão de grupos e geração de relatórios.

### Pipeline de Visão Computacional

Módulo Python que encapsula o modelo YOLO (com fine-tuning) e o tracker (ByteTrack/BoTSORT). Recebe frames individuais, executa detecção + classificação + tracking e retorna a lista de objetos detectados com seus respectivos IDs de tracking, classes, bounding boxes e níveis de confiança.

### Camada de Dados

PostgreSQL (Azure Database) para dados estruturados (grupos, usuários, sessões, eventos de contagem) e Azure Blob Storage para armazenamento de evidências visuais (frames capturados).

## 8. Modelo de Dados

### 8.1 Diagrama Entidade-Relacionamento

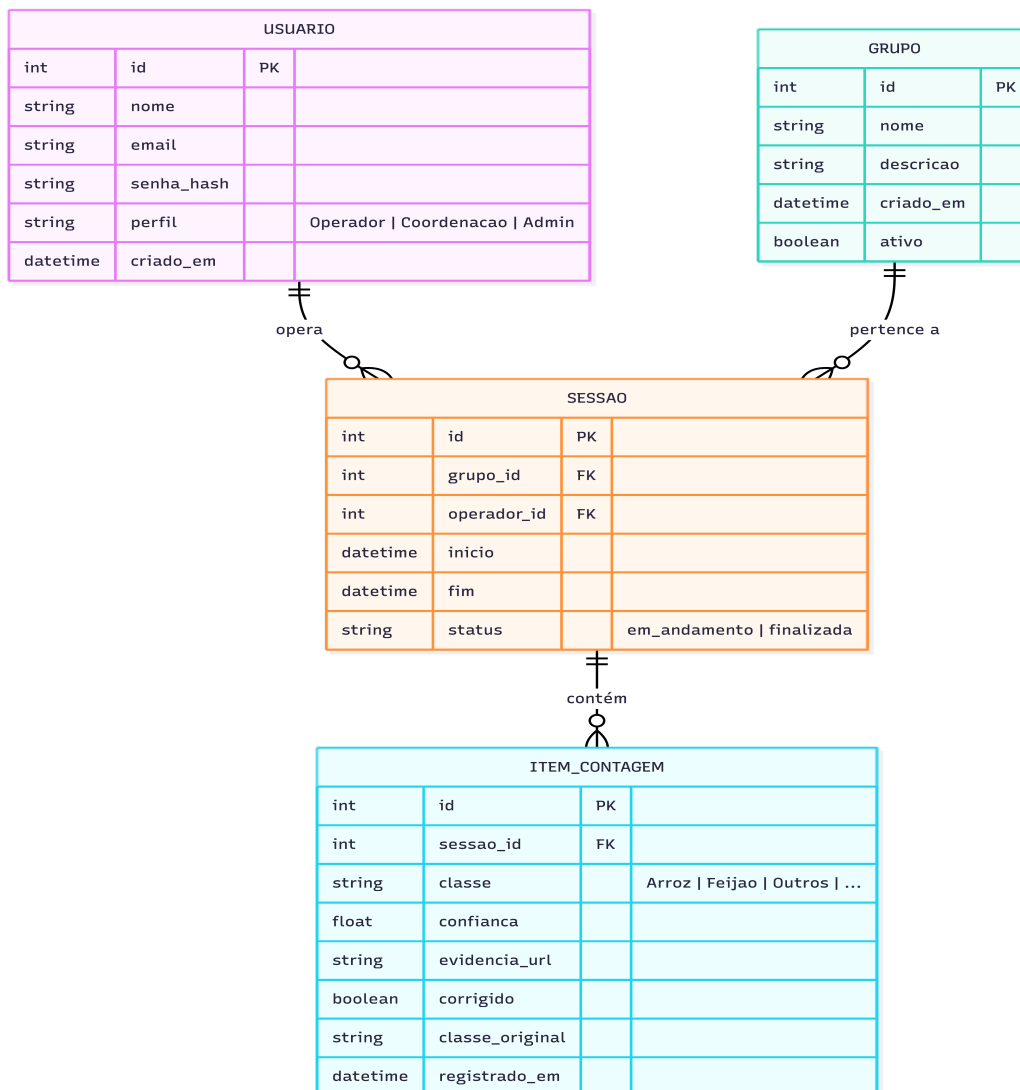


Figura 3: Diagrama entidade-relacionamento do AbraceAI

### 8.2 Descrição das Entidades

#### USUARIO

Representa os usuários do sistema com seus perfis de acesso. A senha é armazenada como hash (bcrypt) para segurança.

#### GRUPO

Equipe arrecadadora cadastrada no sistema. O campo "ativo" permite desativar grupos sem removê-los do histórico.

#### SESSAO

Registro de cada sessão de contagem. Vincula um operador a um grupo em um período específico. O status controla se a sessão ainda está em andamento ou foi finalizada.

#### ITEM\_CONTAGEM

---

Cada item individual registrado pelo scanner. Armazena a classe predita, o nível de confiança, a URL da evidência visual, e se houve correção manual (com a classe original preservada para análise de erros do modelo).

## 9. Pipeline de Visão Computacional

### 9.1 Visão Geral do Pipeline

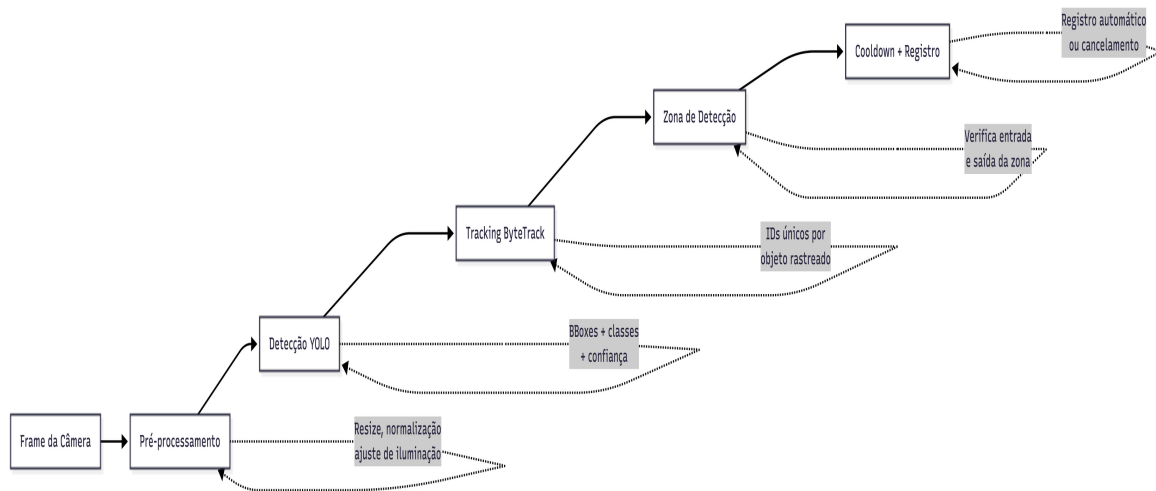


Figura 4: Pipeline de Visão Computacional

### 9.2 Classes de Reconhecimento

Para o MVP, o modelo reconhecerá três classes base. As categorias serão expandidas conforme orientação dos professores. A arquitetura do modelo (YOLO) permite adicionar novas classes com retreinamento incremental do dataset.

| Classe | Descrição                           | Exemplos                                                          |
|--------|-------------------------------------|-------------------------------------------------------------------|
| Arroz  | Pacotes de arroz embalados          | Arroz branco, integral, parboilizado (diversas marcas e tamanhos) |
| Feijão | Pacotes de feijão embalados         | Feijão carioca, preto, branco (diversas marcas e tamanhos)        |
| Outros | Demais itens alimentícios embalados | Macarrão, óleo, leite, enlatados, açúcar, farinha, etc.           |

### 9.3 Modelo de Detecção

O modelo base escolhido é o YOLOv8 (Ultralytics), pela excelente relação entre velocidade e acurácia, ampla documentação e facilidade de fine-tuning. A abordagem de treinamento segue os seguintes passos:

- Utilizar pesos pré-treinados no COCO dataset como ponto de partida.
- Coletar e rotular um dataset próprio com imagens de pacotes de alimentos em cenários representativos (variações de marca, tamanho, orientação, iluminação).
- Realizar fine-tuning do modelo com o dataset próprio, ajustando para as classes definidas.
- Avaliar e iterar o modelo com base nas métricas de validação.

### Tecnologias do Pipeline

| Tecnologia           | Função                                                        |
|----------------------|---------------------------------------------------------------|
| Python               | Linguagem principal do backend e do pipeline de VC.           |
| OpenCV (cv2)         | Captura, pré-processamento de frames e manipulação de imagem. |
| Ultralytics (YOLOv8) | Framework de detecção de objetos para treino e inferência.    |

|                     |                                                                               |
|---------------------|-------------------------------------------------------------------------------|
| ByteTrack / BoTSORT | Algoritmos de tracking multi-objeto integrados ao Ultralytics.                |
| PyTorch             | Framework de deep learning subjacente ao YOLO para treino e fine-tuning.      |
| scikit-learn        | Cálculo de métricas de avaliação (precision, recall, F1, matriz de confusão). |

## 9.4 Estratégia de Contagem sem Duplicidade

A contagem sem duplicidade é um dos desafios centrais do AbraceAI. A estratégia combina três mecanismos complementares para garantir que cada item físico seja contado exatamente uma vez.

### Mecanismo 1: Tracking de Objetos

O algoritmo ByteTrack (integrado ao Ultralytics) atribui um ID único a cada objeto detectado e o acompanha entre frames consecutivos. Isso permite distinguir objetos individuais mesmo quando múltiplos itens aparecem simultaneamente.

### Mecanismo 2: Zona de Detecção

Uma região retangular é definida na interface como "zona de detecção" (overlay visual na tela). Somente itens detectados dentro dessa zona são considerados para contagem. Isso evita que objetos no fundo ou na periferia sejam contabilizados acidentalmente.

### Mecanismo 3: Cooldown Temporal

Após um item ser considerado estável (permaneceu na zona por um período mínimo ou saiu da zona), um timer de cooldown é iniciado. Durante esse período, o item é exibido na interface com seus dados para conferência. O registro só ocorre após o cooldown expirar sem cancelamento do operador. Isso adiciona uma camada final de proteção contra registros acidentais.

## 10. Telas

### 10.1 Wireframes criados na lousa da sala

Os wireframes abaixo foram elaborados pelo time durante sessão de planejamento presencial. O rascunho na lousa captura as ideias iniciais de layout para a Home Page (Kanban dos grupos) e a Tela do Scanner, além da estrutura geral de navegação e dos componentes de backend.

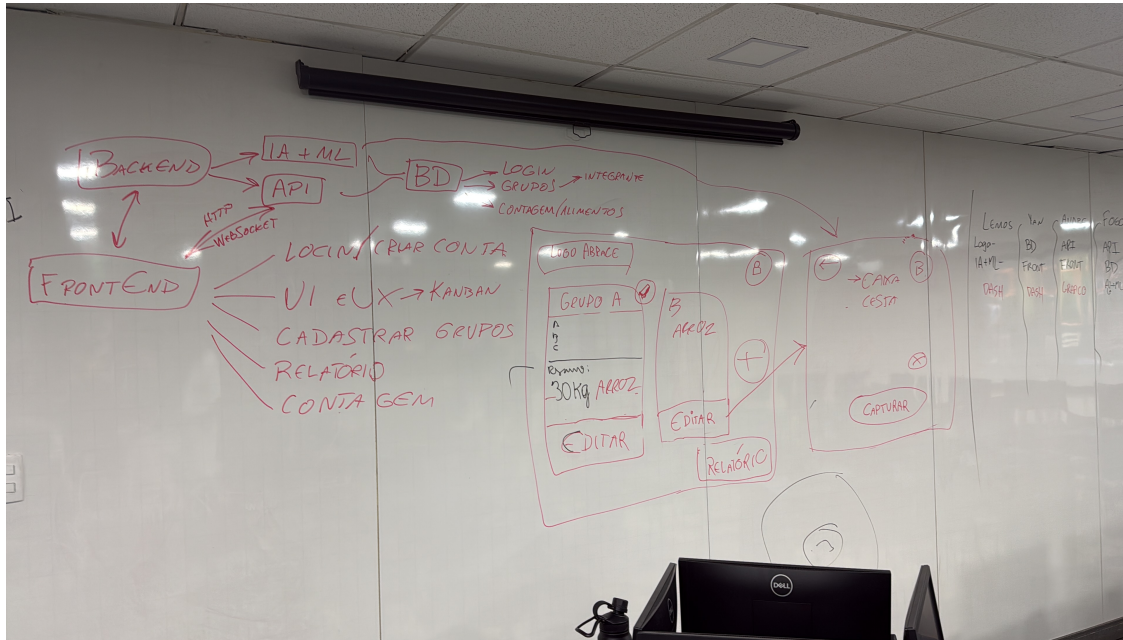


Figura 5: Wireframes elaborados pelo time na lousa durante sessão de planejamento

### 10.2 Protótipos feitos em HTML + CSS

Os protótipos abaixo foram desenvolvidos previamente utilizando a ferramenta Stitch by Google. Excelente para entendermos o valor que iremos entregar visualmente (UI e UX).

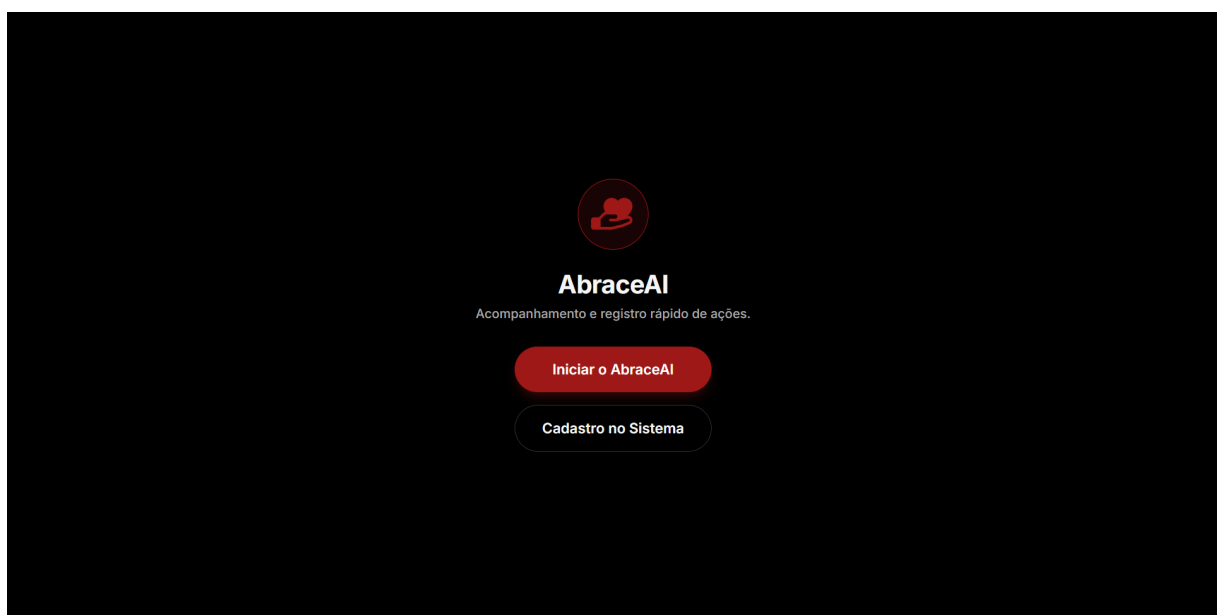


Figura 6: Tela inicial



←

Criar Conta

Nome

Ex: João

Sobrenome

Ex: Silva

E-mail

Ex: joao@email.com

Senha

Crie uma senha forte

Concluir Cadastro

Figura 7: Tela de cadastro

AbraceAI

Relatório

Doações

Pressione Iniciar Contagem nos grupos para abrir a Câmera

Grupo A

Maria Silva (12345) João P. (54321)

Aguardando contagem...

Okkg TOTAL

Iniciar Contagem

Grupo B

Sem integrantes

Aguardando contagem...

Okkg TOTAL

Iniciar Contagem

+

Novo Grupo

Adicionar nova esteira de coleta ao Kanban

Figura 8: Tela no estilo 'Kanban' de identificações do Grupo. Home page.

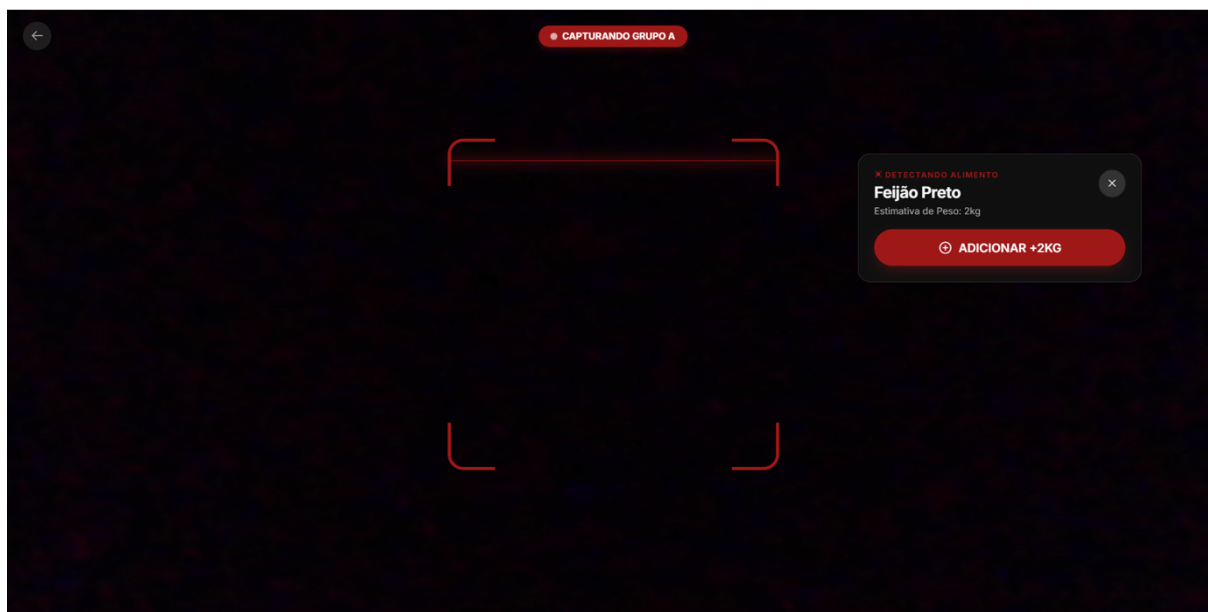


Figura 9: Tela de detecção de alimentos.

## 11. Métricas e Critérios de Validação

### 11.1 Métricas do Modelo de IA

As seguintes métricas serão utilizadas para avaliar o desempenho do modelo de detecção e classificação ao longo do desenvolvimento:

| Métrica                | Descrição                                                                             | Aplicação                                                                        |
|------------------------|---------------------------------------------------------------------------------------|----------------------------------------------------------------------------------|
| Acurácia               | Proporção de predições corretas sobre o total de predições.                           | Visão geral do desempenho; insuficiente isoladamente em datasets desbalanceados. |
| Precision              | Proporção de verdadeiros positivos entre todas as predições positivas de uma classe.  | Mede a confiabilidade das detecções.                                             |
| Recall                 | Proporção de verdadeiros positivos entre todos os exemplos reais de uma classe.       | Mede a cobertura do modelo.                                                      |
| F1-Score               | Média harmônica entre Precision e Recall.                                             | Indicador equilibrado por classe, útil com desbalanceamento.                     |
| mAP                    | Média da Average Precision para cada classe, considerando diferentes limiares de IoU. | Métrica padrão para modelos de detecção de objetos (YOLO).                       |
| Latência de Inferência | Tempo médio (em milissegundos) para processar um frame completo.                      | Fundamental para garantir operação em tempo real.                                |

### 11.2 Métricas de Negócio

| Métrica                  | Descrição                                                                                                                                                                         |
|--------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Taxa de Contagem Correta | Proporção de itens contados corretamente (classe certa + sem duplicidade) sobre o total de itens que passaram pelo scanner. Métrica end-to-end que valida o sistema como um todo. |
| Taxa de Correção Manual  | Proporção de itens que precisaram de correção pelo operador. Indica a confiabilidade prática do modelo em cenário real.                                                           |

### 11.3 Critérios de Validação

- As métricas serão avaliadas em um conjunto de teste separado (nunca utilizado durante o treinamento).
- O dataset de teste deve conter exemplos representativos de todas as classes, com variações de marca, tamanho e condição de iluminação.
- Os resultados serão documentados e comparados entre iterações do modelo para evidenciar a evolução do desempenho.
- A latência de inferência será medida em hardware representativo do ambiente de produção.

## 12. Planejamento Técnico

### 12.1 Tecnologias Escolhidas

| Camada         | Tecnologia                    | Justificativa                                                                                          |
|----------------|-------------------------------|--------------------------------------------------------------------------------------------------------|
| Frontend       | React + Vite                  | Framework moderno com ecossistema robusto, ideal para interfaces interativas e operação em tempo real. |
| Estilização    | Tailwind CSS + Shadcn/UI      | Produtividade alta com componentes prontos e customizáveis.                                            |
| Estado (Front) | Zustand + TanStack Query      | Gerenciamento de estado global leve + cache inteligente de requisições HTTP.                           |
| Kanban         | @hello-pangea/dnd             | Biblioteca de drag-and-drop para a interface Kanban dos grupos.                                        |
| Real-time      | Socket.io (client + server)   | WebSocket para streaming de frames e recebimento de detecções em tempo real.                           |
| Gráficos       | Recharts                      | Visualização de dados nos relatórios e dashboards.                                                     |
| Geração de PDF | @react-pdf/renderer           | Geração de relatórios em PDF diretamente no frontend.                                                  |
| Backend        | FastAPI (Python)              | Framework assíncrono de alta performance com tipagem forte.                                            |
| Modelo de VC   | YOLOv8 (Ultralytics)          | Deteção de objetos estado-da-arte com fine-tuning e tracker integrado.                                 |
| Tracking       | ByteTrack / BoTSORT           | Tracking multi-objeto integrado ao Ultralytics para anti-duplicidade.                                  |
| Imagem         | OpenCV (cv2)                  | Pré-processamento de frames e manipulação de imagem.                                                   |
| Deep Learning  | PyTorch                       | Framework subjacente ao YOLO, utilizado para treino e fine-tuning.                                     |
| Métricas       | scikit-learn                  | Cálculo de precision, recall, F1-score e matrizes de confusão.                                         |
| Banco de Dados | PostgreSQL (Azure)            | Banco relacional robusto para dados estruturados e queries complexas.                                  |
| Storage        | Azure Blob Storage            | Armazenamento escalável de evidências visuais.                                                         |
| Deploy         | Docker + Azure Container Apps | Containerização e deploy em serviço gerenciado.                                                        |
| CI/CD          | GitHub Actions                | Automação de testes, build e deploy contínuo.                                                          |
| Versionamento  | Git + GitHub                  | Controle de versão e colaboração do time.                                                              |

## 12.2 Roadmap de Desenvolvimento

O cronograma abaixo distribui as atividades ao longo de 13 semanas, organizadas por frente de trabalho.

| Período       | Foco                | Atividades                                                                                                              |
|---------------|---------------------|-------------------------------------------------------------------------------------------------------------------------|
| Semanas 1-2   | Planejamento        | Entendimento do problema com o LE, definição do MVP, requisitos e desenho do fluxo de operação.                         |
| Semanas 3-4   | Dataset e Modelo    | Prototipação do ambiente (câmera, iluminação), coleta inicial e rotulagem de dados, baseline de VC (primeiro modelo).   |
| Semanas 5-6   | Backend + Modelo    | Treinamento e fine-tuning do modelo, backend mínimo (API + DB), integração de inferência + WebSocket, Home Page Kanban. |
| Semanas 7-8   | Integração          | Melhoria do modelo e métricas, contagem sem duplicidade e evidências, Tela do Scanner, integração real-time.            |
| Semanas 9-10  | Relatórios + Testes | Relatórios por equipe/categoria, histórico, testes de bancada com métricas e ajustes de desempenho.                     |
| Semanas 11-12 | Deploy              | Deploy em nuvem (Azure), documentação final, testes finais e preparação.                                                |
| Semana 13     | Entrega             | Entrega final e apresentação com demonstração do sistema.                                                               |